



CE 222 – Computer Organization and Assembly Language (COAL)

Lecture 3

Dr. Asad Mahmood

Jan. 23, 2026

Lecture Outline

- Brief Outline to Previous Lecture:
 - **Chapter 1 - Computer Abstractions and Technology**
 - 1.1 Introduction
 - 1.2 Seven Great Ideas in Computer Architecture
 - 1.3 Below Your Program
- Outline to Today's Lecture:
 - **Chapter 1 - Computer Abstractions and Technology**
 - 1.4 Under the Covers
 - 1.5 Technologies for Building Processors and Memory
 - 1.6 Performance
 - 1.7 The Power Wall

COMPUTER ORGANIZATION AND DESIGN RISC-V EDITION

THE HARDWARE SOFTWARE INTERFACE

SECOND EDITION

From Wikipedia, the free encyclopedia

John Leroy Hennessy (born 22 September, 1952) is an American computer scientist and chairman of [Alphabet Inc.](#)^[8] Hennessy is one of the founders of [MIPS Technologies](#) and [Atheros](#), serving as 10th president of [Stanford University](#) from 2000 to 2016. He was succeeded as president by [Marc Tessier-Lavigne](#).^[9] [Marc Andreessen](#) called him "the godfather of [Silicon Valley](#)."^[10]

Along with [David Patterson](#), Hennessy was a recipient of the 2017 [Turing Award](#) for their work in developing the [reduced instruction set computer](#) (RISC) architecture, which is now used in 99% of new computer chips.^{[11][12][13][14][15][16][17]}

Early life and education [\[edit\]](#)

Hennessy was raised in [Huntington, New York](#), as one of six children.^[10] His father was an aerospace engineer, and his mother was a teacher before raising her children.^[10] He is of Irish-Catholic descent, with some of his ancestors arriving in America during the potato famine in the 19th century.^[18]

He earned his [bachelor's degree](#) in [electrical engineering](#) from [Villanova University](#), and his [master's degree](#) and [Doctor of Philosophy](#) in [computer science](#) from [Stony Brook University](#).^{[6][19][20]}

John L. Hennessy



Hennessy in 2024

10th President of [Stanford University](#)

In office

September 1, 2000^[1] – August 31, 2016

Preceded by [Gerhard Casper](#)

Succeeded by [Marc Tessier-Lavigne](#)

11th Provost of [Stanford University](#)

David Patterson (computer scientist)

Article Talk

Read Edit View history Tools

From Wikipedia, the free encyclopedia

This article is about the American computer scientist. For other spellings and people of the same name, see [David Patterson \(disambiguation\)](#).

David Andrew Patterson (born November 16, 1947) is an American [computer scientist](#) and academic who has held the position of professor of [computer science](#) at the [University of California, Berkeley](#) since 1976. He is a [computer pioneer](#). He announced retirement in 2016 after serving nearly forty years, becoming a distinguished software engineer at [Google](#).^{[5][6]} He currently is vice chair of the board of directors of the [RISC-V Foundation](#),^[7] and the Pardee Professor of Computer Science, [Emeritus](#) at UC Berkeley.^[8]

Patterson is noted for his pioneering contributions to [reduced instruction set computer](#) (RISC) design, having coined the term RISC, and by leading the [Berkeley RISC](#) project.^[9] As of 2018, 99% of all new chips use a RISC architecture.^{[10][11]} He is also noted for leading the research on redundant arrays of inexpensive disks ([RAID](#)) storage, with [Randy Katz](#).^[12]

His books on [computer architecture](#), co-authored with [John L. Hennessy](#), are widely used in [computer science education](#). Hennessy and Patterson won the 2017 [Turing Award](#) for their work in developing RISC.

Early life and education [\[edit \]](#)

David Patterson grew up in [Evergreen Park, Illinois](#). He graduated from [South High](#)

David Patterson



Born	David Andrew Patterson November 16, 1947 (age 78) Evergreen Park, Illinois, U.S.
Education	University of California, Los Angeles (BA, MS, PhD)
Known for	Reduced instruction set computer RAID Network of Workstations
Spouse	Linda Patterson (1967)
Awards	Charles Stark Draper Prize (2022)

Authors First Word

The most beautiful thing we can experience is the mysterious. It is the source of all true art and science.

Albert Einstein, *What I Believe*, 1930

- Modern computer technology requires professionals of every computing specialty to understand both hardware and software.
- Our view is that for at least the next decade, most programmers are going to have to understand the hardware/software interface if they want programs to run efficiently on parallel computers.

Chapter 1

Computer Abstractions and Technology

Chapter Outline

- **1 Computer Abstractions and Technology**
 - [1.1 Introduction](#)
 - 1.2 Seven Great Ideas in Computer Architecture
 - 1.3 Below Your Program
 - 1.4 Under the Covers
 - 1.5 Technologies for Building Processors and Memory
 - 1.6 Performance
 - 1.7 The Power Wall
 - 1.8 The Sea Change: The Switch from Uniprocessors to Multiprocessors
 - 1.9 Real Stuff: Benchmarking the Intel Core i7
 - 1.10 Going Faster: Matrix Multiply in Python
 - 1.11 Fallacies and Pitfalls
 - 1.12 Concluding Remarks
 - 1.13 Historical Perspective and Further Reading

The Computer Revolution

- Progress in computer technology
 - Comparison with transportation industry
- Makes novel applications feasible -> Science fiction became reality!
 - Computers in automobiles
 - Cell phones
 - Human genome project
 - World Wide Web
 - Search Engines
- Computers are pervasive

Classes of Computers

1. Personal computers

- General purpose, variety of software
- Subject to cost/performance tradeoff

2. Server computers

- Network based
- High capacity, performance, reliability
- Range from small servers to building sized

Classes of Computers

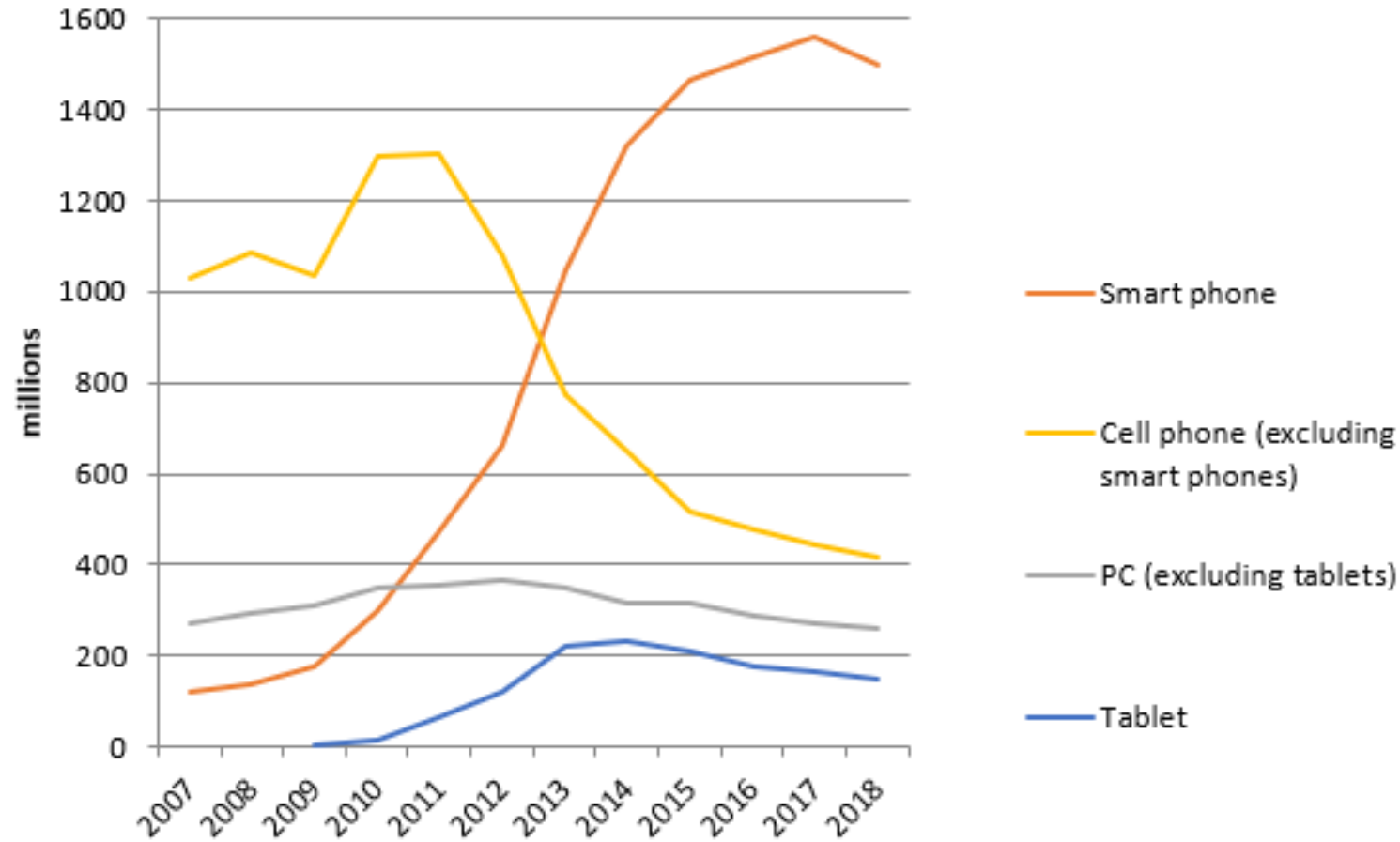
3. Supercomputers

- Type of server
- High-end scientific and engineering calculations
- Highest capability but represent a small fraction of the overall computer market
- Applications?

4. Embedded computers

- Hidden as components of systems
- Stringent power/performance/cost constraints
- IoT
- Not the focus of this book, but same principles apply!

The PostPC Era



The PostPC Era

- Personal Mobile Device (PMD)
 - Battery operated
 - Connects to the Internet
 - Hundreds of dollars
 - Smart phones, tablets, electronic glasses
- Cloud computing
 - Warehouse Scale Computers (WSC)
 - Software as a Service (SaaS)
 - Portion of software run on a PMD and a portion run in the Cloud
 - Amazon and Google

What You Will Learn in this Book

- How programs are translated into the machine language?
 - And how the hardware executes them?
- The hardware/software interface
- What determines program performance?
 - And how it can be improved?
- How hardware designers improve performance?
- What is parallel processing?

Understanding Performance

Computer performance depends on all of the following:

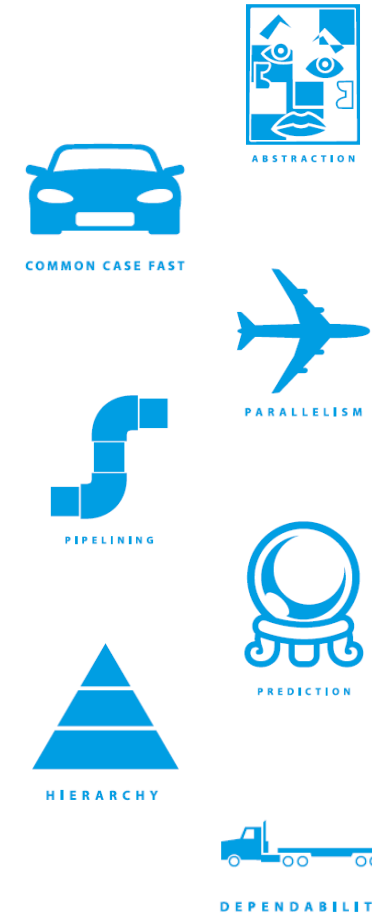
1. Algorithm
 - Determines number of operations executed
2. Programming language, compiler, architecture
 - Determine number of machine instructions executed per operation
3. Processor and memory system
 - Determine how fast instructions are executed
4. I/O system (including OS)
 - Determines how fast I/O operations are executed

Chapter Outline

- **1 Computer Abstractions and Technology**
 - 1.1 Introduction
 - [1.2 Seven Great Ideas in Computer Architecture](#)
 - 1.3 Below Your Program
 - 1.4 Under the Covers
 - 1.5 Technologies for Building Processors and Memory
 - 1.6 Performance
 - 1.7 The Power Wall
 - 1.8 The Sea Change: The Switch from Uniprocessors to Multiprocessors
 - 1.9 Real Stuff: Benchmarking the Intel Core i7
 - 1.10 Going Faster: Matrix Multiply in Python
 - 1.11 Fallacies and Pitfalls
 - 1.12 Concluding Remarks
 - 1.13 Historical Perspective and Further Reading

Seven Great Ideas which have revolutionized Computing

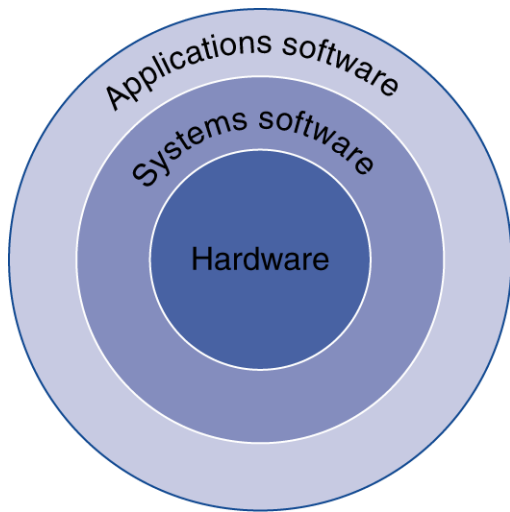
1. Use **abstraction** to simplify design
2. Make the **common case fast**
3. Performance via **parallelism**
4. Performance via **pipelining**
5. Performance via **prediction**
6. **Hierarchy** of memories
7. **Dependability** via redundancy



Chapter Outline

- **1 Computer Abstractions and Technology**
 - 1.1 Introduction
 - 1.2 Seven Great Ideas in Computer Architecture
 - [1.3 Below Your Program](#)
 - 1.4 Under the Covers
 - 1.5 Technologies for Building Processors and Memory
 - 1.6 Performance
 - 1.7 The Power Wall
 - 1.8 The Sea Change: The Switch from Uniprocessors to Multiprocessors
 - 1.9 Real Stuff: Benchmarking the Intel Core i7
 - 1.10 Going Faster: Matrix Multiply in Python
 - 1.11 Fallacies and Pitfalls
 - 1.12 Concluding Remarks
 - 1.13 Historical Perspective and Further Reading

Below Your Program



- Application software
 - Written in high-level language (HLL)
- System software
 - **Compiler**: translates HLL code to machine code
 - **Operating System**: service code
 - Handling input/output
 - Managing memory and storage
 - Scheduling tasks & sharing resources
- Hardware
 - Processor, memory, I/O controllers

Levels of Program Code

- High-level language
 - Level of abstraction closer to problem domain
 - Provides for productivity and portability
- Assembly language
 - Textual representation of instructions
- Hardware representation
 - Binary digits (bits)
 - Encoded instructions and data

High-level
language
program
(in C)

```
swap(int v[], int k)
{int temp;
  temp = v[k];
  v[k] = v[k+1];
  v[k+1] = temp;
}
```

Compiler

Assembly
language
program
(for RISC-V)

```
swap:
  slli x6, x11, 3
  add x6, x10, x6
  ld x5, 0(x6)
  ld x7, 8(x6)
  sd x7, 0(x6)
  sd x5, 8(x6)
  jalr x0, 0(x1)
```

Assembler

Binary machine
language
program
(for RISC-V)

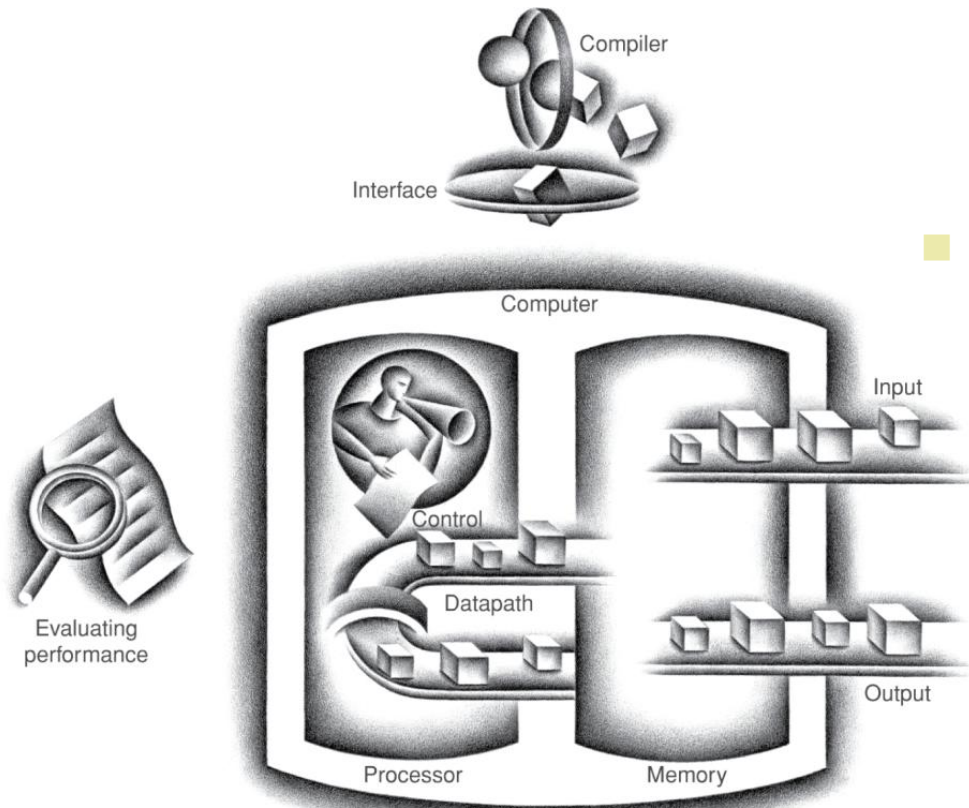
```
00000000001101011001001100010011
000000000011001010000001100110011
000000000000000110011001010000011
00000000100000110011001110000011
00000000011100110011000000100011
00000000010100110011010000100011
0000000000000000100000001100111
```

Chapter Outline

- **1 Computer Abstractions and Technology**
 - 1.1 Introduction
 - 1.2 Seven Great Ideas in Computer Architecture
 - 1.3 Below Your Program
 - [1.4 Under the Covers](#)
 - 1.5 Technologies for Building Processors and Memory
 - 1.6 Performance
 - 1.7 The Power Wall
 - 1.8 The Sea Change: The Switch from Uniprocessors to Multiprocessors
 - 1.9 Real Stuff: Benchmarking the Intel Core i7
 - 1.10 Going Faster: Matrix Multiply in Python
 - 1.11 Fallacies and Pitfalls
 - 1.12 Concluding Remarks
 - 1.13 Historical Perspective and Further Reading

Components of a Computer

The BIG Picture



- Same components for all kinds of computer
 - Desktop, server, embedded
- **Input/output** includes
 - **User-interface devices**
 - Display, keyboard, mouse
 - **Storage devices**
 - Hard disk, CD/DVD, flash
 - **Network adapters**
 - For communicating with other computers

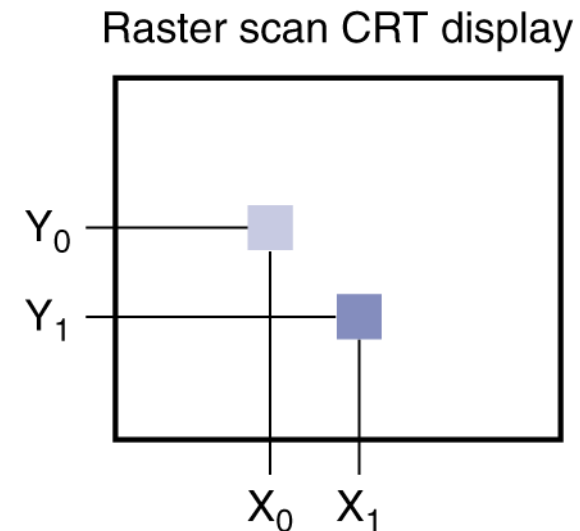
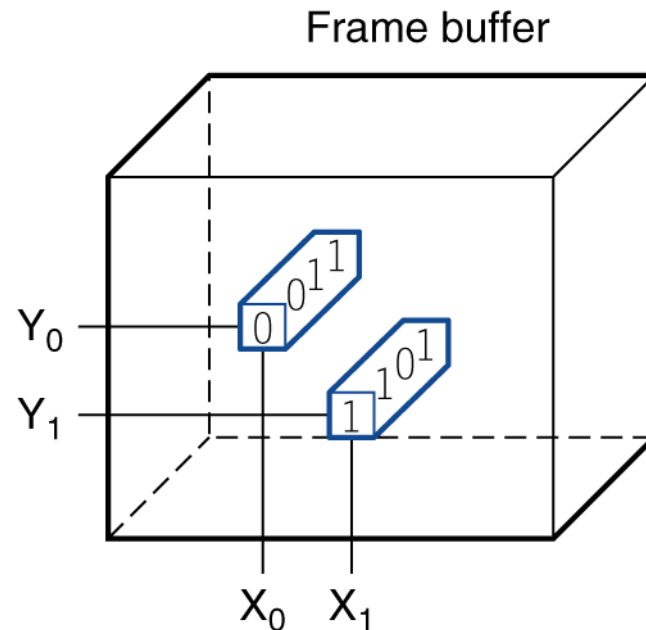
Touchscreen

- PostPC device
- Supersedes keyboard and mouse
- Resistive and Capacitive types
 - Most tablets, smart phones use capacitive
 - Capacitive allows multiple touches simultaneously

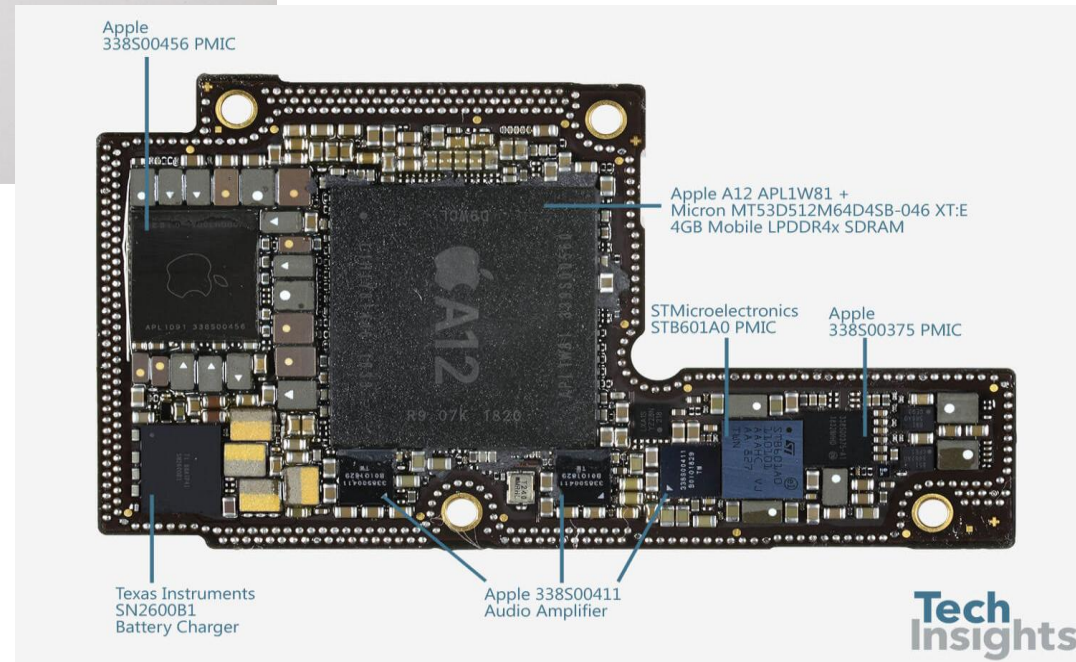
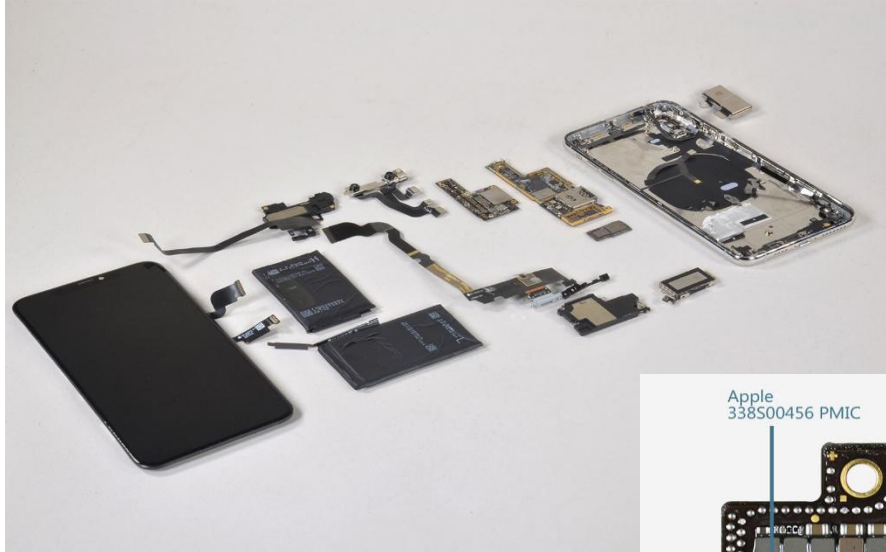


Through the Looking Glass

- LCD screen: picture elements (pixels)
 - Mirrors content of frame buffer memory



Opening the Box

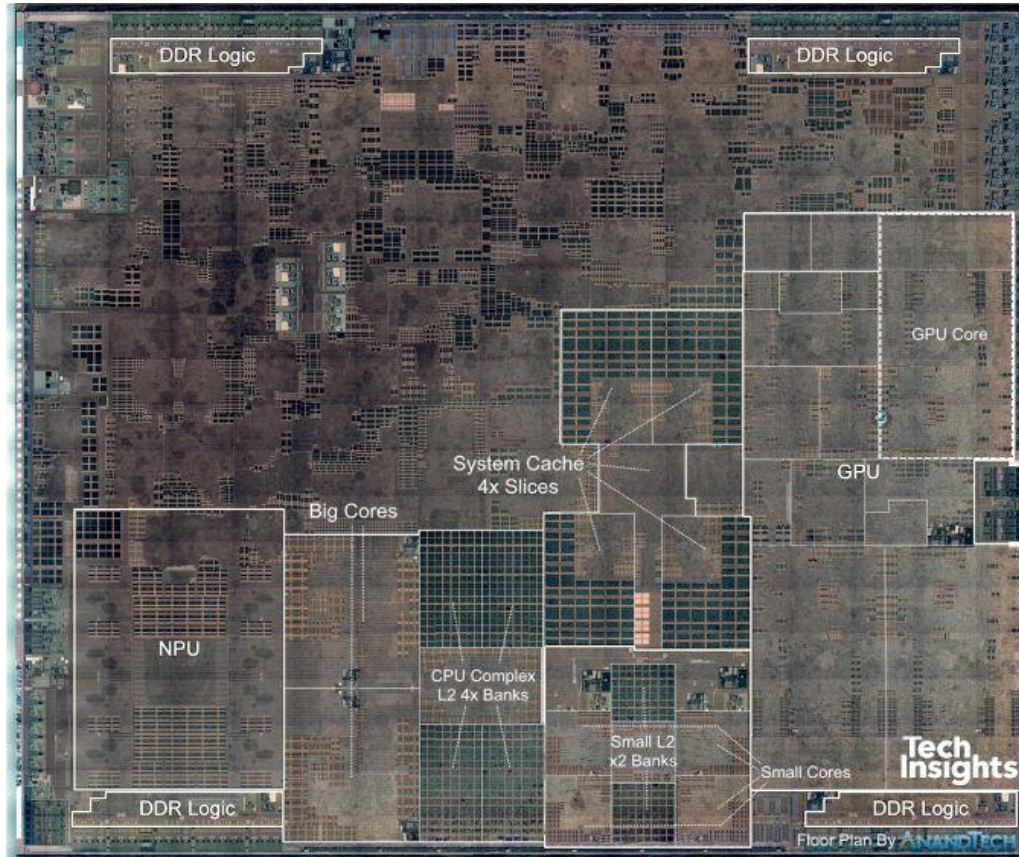


Inside the Processor (CPU)

- **Datapath:** performs operations on data
- **Control:** sequences datapath, memory, ...
- **Cache memory**
 - Small fast SRAM memory for immediate access to data

Inside the Processor

- A12 processor



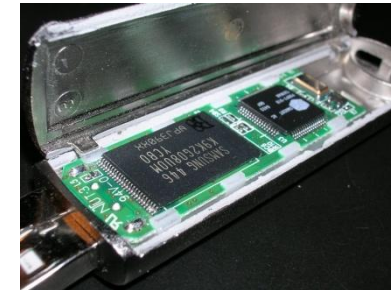
Abstractions

The BIG Picture

- Abstraction helps us deal with complexity
 - Hide lower-level detail
- Instruction set architecture (ISA)
 - The hardware/software interface
- Application binary interface
 - The ISA plus system software interface
- Implementation
 - The details underlying and interface

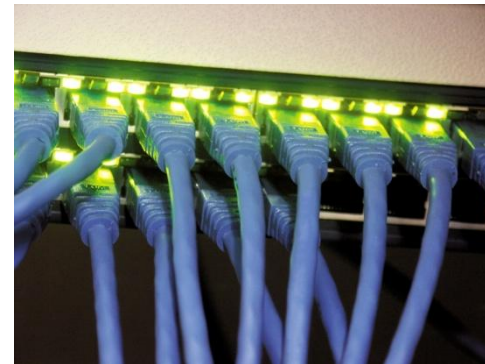
A Safe Place for Data

- Volatile main memory
 - Loses instructions and data when power off
- Non-volatile secondary memory
 - Magnetic disk
 - Flash memory
 - Optical disk (CDROM, DVD)



Networks

- Communication, resource sharing, nonlocal access
- Local area network (LAN): Ethernet
- Wide area network (WAN): the Internet
- Wireless network: WiFi, Bluetooth

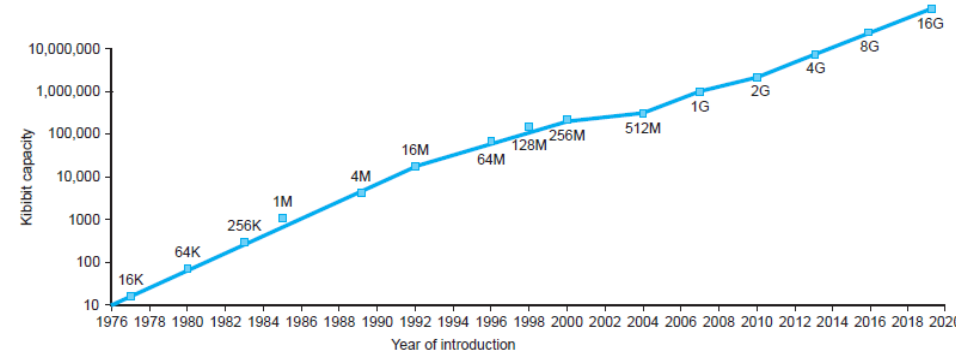


Chapter Outline

- **1 Computer Abstractions and Technology**
 - 1.1 Introduction
 - 1.2 Seven Great Ideas in Computer Architecture
 - 1.3 Below Your Program
 - 1.4 Under the Covers
 - [1.5 Technologies for Building Processors and Memory](#)
 - 1.6 Performance
 - 1.7 The Power Wall
 - 1.8 The Sea Change: The Switch from Uniprocessors to Multiprocessors
 - 1.9 Real Stuff: Benchmarking the Intel Core i7
 - 1.10 Going Faster: Matrix Multiply in Python
 - 1.11 Fallacies and Pitfalls
 - 1.12 Concluding Remarks
 - 1.13 Historical Perspective and Further Reading

Technology Trends

- Electronics technology continues to evolve
 - Increased capacity and performance
 - Reduced cost
 - **Moore's Law**



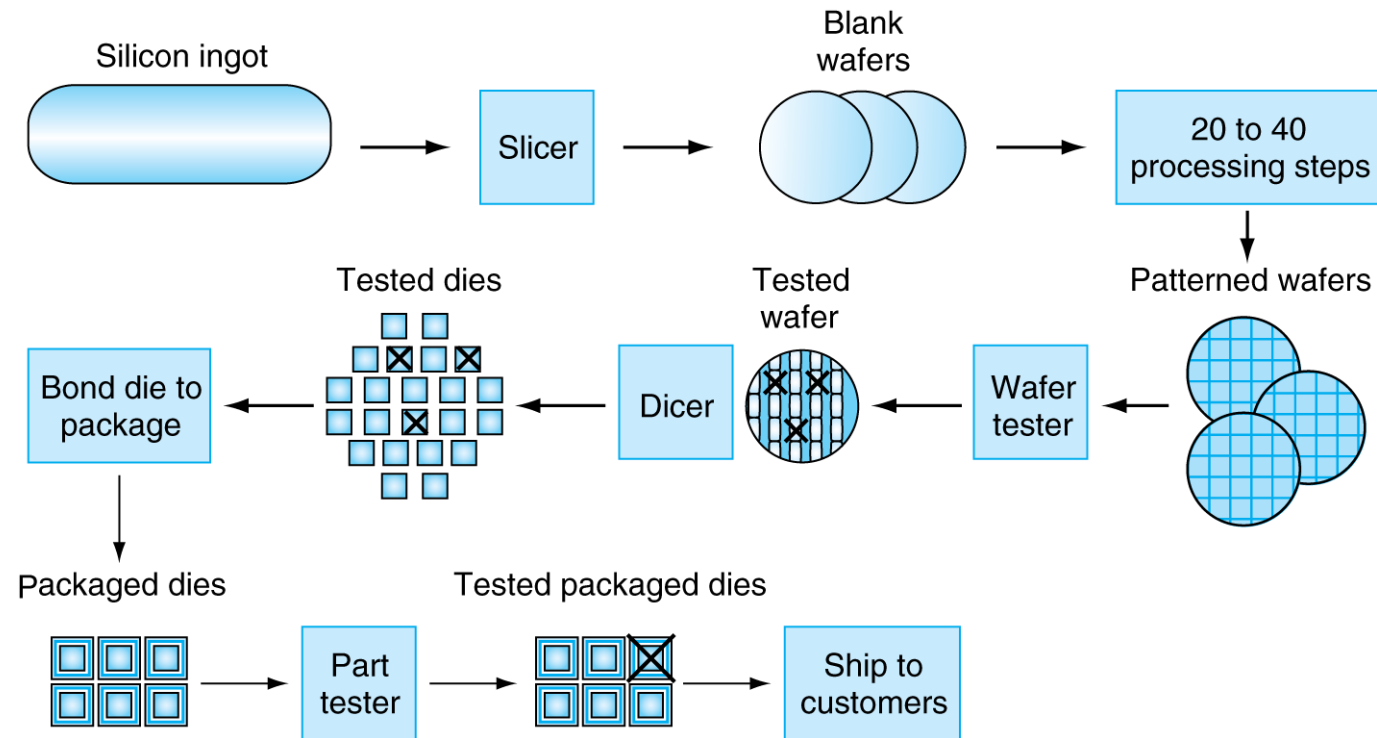
DRAM capacity

Year	Technology	Relative performance/cost
1951	Vacuum tube	1
1965	Transistor	35
1975	Integrated circuit (IC)	900
1995	Very large scale IC (VLSI)	2,400,000
2013	Ultra large scale IC	250,000,000,000

Semiconductor Technology

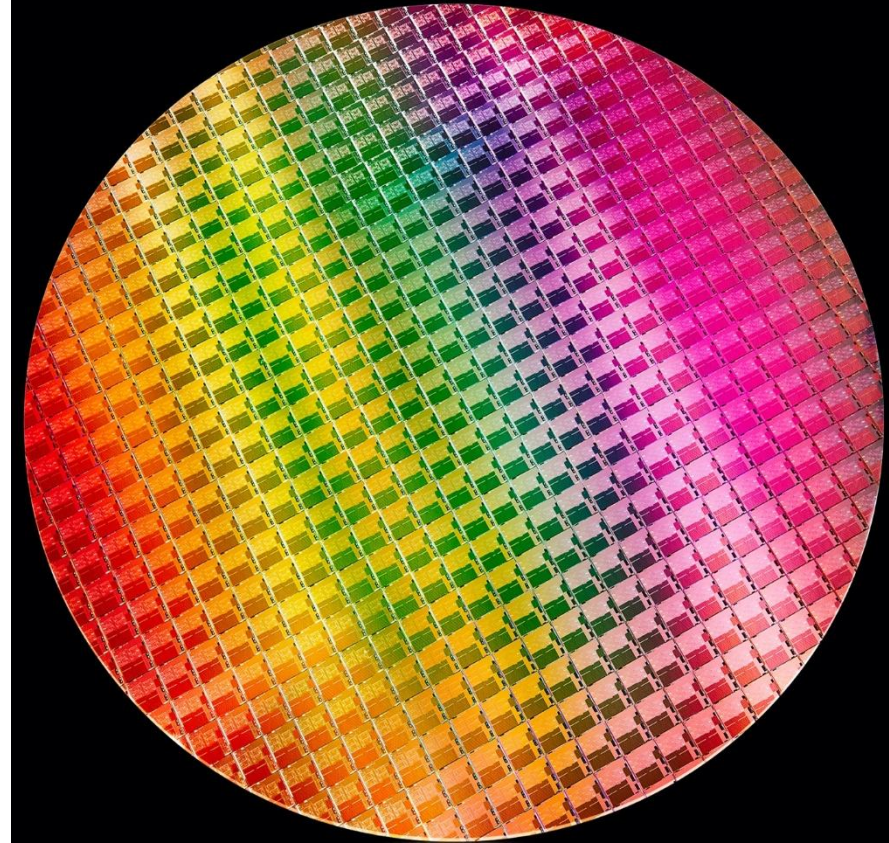
- Silicon: **semiconductor**
- Add materials to transform properties:
 - Conductors
 - Insulators
 - Switch

Manufacturing ICs



- Yield: proportion of working dies per wafer

Intel® Core 10th Gen



- 300mm wafer, 506 chips, 10nm technology
- Each chip is 11.4 x 10.7 mm

Integrated Circuit Cost

$$\text{Cost per die} = \frac{\text{Cost per wafer}}{\text{Dies per wafer} \times \text{Yield}}$$

$$\text{Dies per wafer} \approx \text{Wafer area} / \text{Die area}$$

$$\text{Yield} = \frac{1}{(1 + (\text{Defects per area} \times \text{Die area} / 2))^2}$$

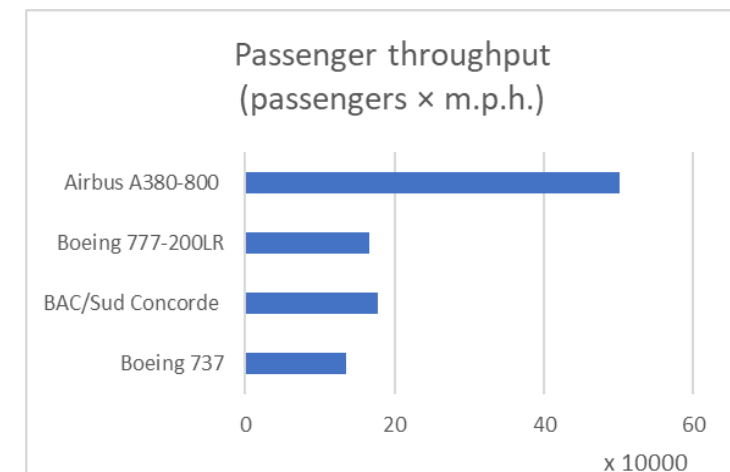
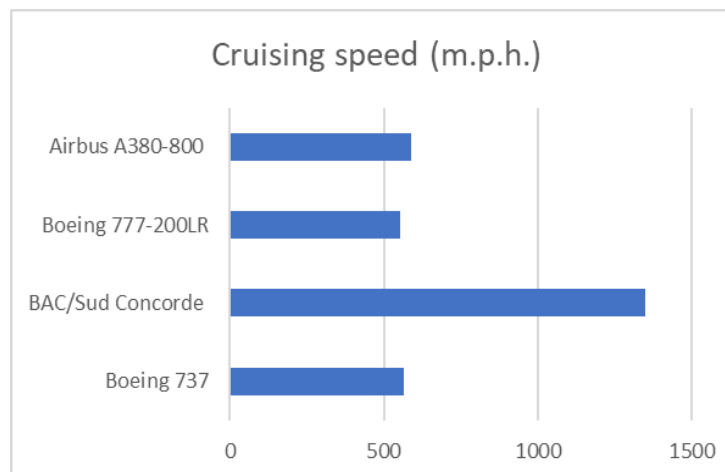
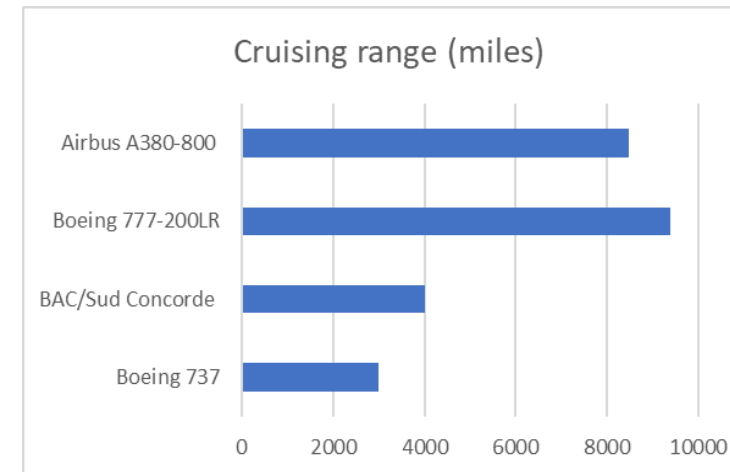
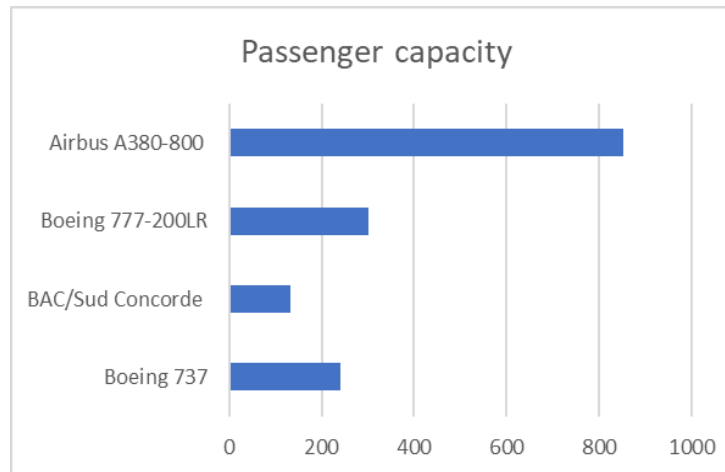
- Nonlinear relation to area and defect rate
 - Wafer cost and area are fixed
 - Defect rate determined by manufacturing process
 - Die area determined by architecture and circuit design

Chapter Outline

- **1 Computer Abstractions and Technology**
 - 1.1 Introduction
 - 1.2 Seven Great Ideas in Computer Architecture
 - 1.3 Below Your Program
 - 1.4 Under the Covers
 - 1.5 Technologies for Building Processors and Memory
 - **1.6 Performance**
 - 1.7 The Power Wall
 - 1.8 The Sea Change: The Switch from Uniprocessors to Multiprocessors
 - 1.9 Real Stuff: Benchmarking the Intel Core i7
 - 1.10 Going Faster: Matrix Multiply in Python
 - 1.11 Fallacies and Pitfalls
 - 1.12 Concluding Remarks
 - 1.13 Historical Perspective and Further Reading

Defining Performance

- Very important to define computer performance but not easy!
- Which airplane has the best performance?



Response Time and Throughput

- We can define computer performance in several distinct ways
- Response time
 - How long it takes to do a task
 - Focus of an individual user
- Throughput
 - Total work done per unit time e.g., Total tasks / transactions/... per hour
 - Focus of a data-center/server
- How are response time and throughput affected by
 1. Case 1 - Replacing the processor with a faster version?
 2. Case 2 - Adding more processors?
- We'll focus on response time for now...

Relative Performance

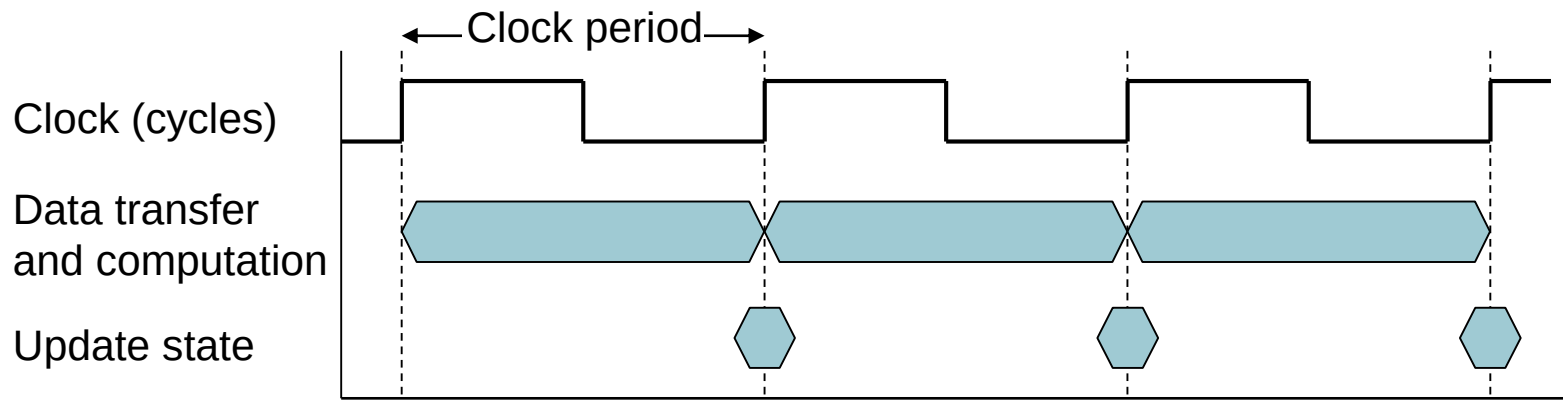
- Let us define the *Performance* of a computer in terms of its Execution Time
- What it means when one says “X is n time faster than Y”?
- **Example:** If computer A runs a program in 10 seconds and computer B runs the same program in 15 seconds, how much faster is A than B?

Measuring Execution Time

- Execution time (Seconds/Program) can be measured in a number of ways, e.g.
 1. Elapsed time = Total response time, including all aspects i.e. Processing, I/O, OS overhead, idle time
 2. CPU time = Time spent processing a given job without including I/O time, other jobs' shares
 - Comprises user CPU time and system CPU time
- Which one of the above two is experienced by the user?
- We will focus on CPU performance in this chapter

CPU Clocking

- Operation of digital hardware governed by a **constant-rate clock**, which **can be used to define CPU time**



- Clock period (s) and clock frequency / rate (cycles/s)
- Relating Clock period/Freq. with CPU time
- How performance can be improved?
- Often a tradeoff involved!